

TP 3 – Sémaphores dans mthread

Programmation à base de threads – S4

21 février 2026

1 Sémaphore – Exemple

Q.1 : Décrire à l'aide d'un schéma l'utilité du sémaphore dans l'exemple précédent

Dans l'exemple q1.c, deux threads (filsA et filsB) exécutent la même fonction `affichage()` qui imprime des caractères ("A" ou "B") sur la sortie standard. Sans sémaphore, les affichages des deux threads s'entremêleraient de manière imprévisible (ex : AAABBBAAAB...).
Le sémaphore `mutex` (initialisé à 1) est utilisé comme **verrou d'exclusion mutuelle** :

Thread A		Thread B
--- <code>sem_wait(&mutex)</code> ----->		(value: 1 -> 0, A entre)
[affiche AAAAA]		
<code>sched_yield()</code>		--- <code>sem_wait(&mutex)</code>
[affiche AAAAA + \n]		(value=0, B BLOQUE)
--- <code>sem_post(&mutex)</code> ----->		
		(veille B)
--- <code>sem_wait(&mutex)</code>		[affiche BBBBB]
(value=0, A BLOQUE)		<code>sched_yield()</code>
		[affiche BBBBB + \n]
		--- <code>sem_post(&mutex)</code> (veille
A)		
...	...	...

Utilité : Le sémaphore garantit que chaque ligne complète (10 caractères identiques + \n) est affichée **de manière atomique**, sans entrelacement. Même si `sched_yield()` est appelé au milieu de l'affichage, le thread B ne peut pas intervenir car il est bloqué sur `sem_wait`. Cela assure que la sortie sera toujours des lignes de AAAAAAAAAA ou BBBBBBBBBB, jamais mélangées.

2 Mise en place des sémaphores dans mthread

Structure du sémaphore

La structure `mthread_sem_t` a été définie dans `mthread.h` :

```

1 typedef struct mthread_sem_s {
2     bool is_initialized = false;           // Indique si le semaphore est
        initialise
3     atomic_int value = 0;                  // Compteur atomique du semaphore
4     atomic_flag thread_list_spinlock = ATOMIC_FLAG_INIT; // Spinlock
5     mthread_list_t thread_list;           // Liste des threads en attente
6 } mthread_sem_t;

```

- `is_initialized` – Drapeau indiquant si le sémaphore a été initialisé.
- `value` – Compteur atomique représentant la valeur du sémaphore.
- `thread_list_spinlock` – Verrou spinlock protégeant l'accès concurrent à la liste d'attente.
- `thread_list` – Liste chaînée des threads bloqués en attente sur le sémaphore.

Q.2 : mthread_sem_init

Rôle : Initialise un sémaphore avec une valeur donnée.

```

1 int mthread_sem_init(mthread_sem_t *sem, int value) {
2     if (sem == nullptr) {
3         return MTHREAD_SEM_ERROR_NULL;
4     }
5     if (value < 0) {
6         return MTHREAD_SEM_ERROR_INIT;
7     }
8     if (sem->is_initialized) {
9         return MTHREAD_SEM_ERROR_INIT;
10    }
11    sem->is_initialized = true;
12    atomic_store(&(sem->value), value);
13    sem->thread_list_spinlock = (atomic_flag)ATOMIC_FLAG_INIT;
14    sem->thread_list.head = nullptr;
15    sem->thread_list.tail = nullptr;
16    return 0;
17 }

```

Détails :

- On vérifie que le pointeur `sem` n'est pas NULL.
- On refuse les valeurs négatives (`MTHREAD_SEM_ERROR_INIT`).
- On refuse une double initialisation (`MTHREAD_SEM_ERROR_INIT`).
- On stocke la valeur de manière atomique et on initialise le spinlock et la liste d'attente à vide.

Q.3 : mthread_sem_wait

Rôle : Décrémente le sémaphore. Si la valeur est 0, le thread appelant est bloqué jusqu'à ce qu'un autre thread fasse `sem_post`.

```

1 int mthread_sem_wait(mthread_sem_t *sem) {
2     if (sem == nullptr) {
3         return MTHREAD_SEM_ERROR_NULL;
4     }

```

```

5  if (!sem->is_initialized) {
6      return MTHREAD_SEM_ERROR_INIT;
7  }
8
9  mthread_internal_spin_lock(&(sem->thread_list_spinlock));
10
11  if (atomic_load(&(sem->value)) > 0) {
12      atomic_fetch_sub(&(sem->value), 1);
13      mthread_internal_spin_unlock(&(sem->thread_list_spinlock));
14      return 0;
15  }
16
17  // Valeur == 0 : on bloque le thread
18  mthread_thread_t *current_thread = mthread_self();
19  mthread_list_item_t item;
20  item.thread = current_thread;
21  insert_tail_in_list(&item, &sem->thread_list);
22  mthread_block_self(&(sem->thread_list_spinlock));
23
24  return 0;
25 }

```

Détails :

- On prend le spinlock pour protéger l'accès concurrent à `value` et à la liste d'attente.
- Si `value > 0` : on décrémente et on libère le spinlock → le thread continue.
- Si `value == 0` : on insère le thread courant dans la liste d'attente (`thread_list`) puis on appelle `mthread_block_self` qui bloque le thread **et** libère le spinlock de manière atomique (même mécanisme que le mutex).

Q.4 : mthread_sem_post

Rôle : Incrémente le sémaphore. Si des threads sont en attente, en réveille un.

```

1  int mthread_sem_post(mthread_sem_t *sem) {
2      if (sem == nullptr) {
3          return MTHREAD_SEM_ERROR_NULL;
4      }
5      if (!sem->is_initialized) {
6          return MTHREAD_SEM_ERROR_INIT;
7      }
8
9      mthread_internal_spin_lock(&(sem->thread_list_spinlock));
10
11      if (sem->thread_list.head != nullptr) {
12          mthread_list_item_t *item = remove_head_in_list(&(sem->thread_list))
13          ;
14          mthread_internal_spin_unlock(&(sem->thread_list_spinlock));
15          mthread_wake_thread(item->thread);
16      } else {
17          atomic_fetch_add(&(sem->value), 1);
18          mthread_internal_spin_unlock(&(sem->thread_list_spinlock));
19      }
20
21      return 0;
22  }

```

Détails :

- On prend le spinlock.
- S'il y a des threads bloqués dans `thread_list` : on retire le premier de la liste (FIFO) et on le réveille avec `pthread_wake_thread`. La valeur du sémaphore ne change pas (le jeton est transféré directement au thread réveillé).
- S'il n'y a pas de thread en attente : on incrémente simplement la valeur.

Q.5 : pthread_sem_destroy

Note : La fonction `pthread_sem_destroy` n'est pas déclarée dans l'API `pthread.h` de ce TD. Voici comment elle pourrait être implémentée par analogie avec `pthread_mutex_destroy` :

```

1 int pthread_sem_destroy(pthread_sem_t *sem) {
2     if (sem == nullptr) {
3         return PTHREAD_SEM_ERROR_NULL;
4     }
5     if (!sem->is_initialized) {
6         return PTHREAD_SEM_ERROR_INIT;
7     }
8     // On ne peut pas détruire un sémaphore si des threads sont en attente
9     if (sem->thread_list.head != nullptr) {
10        return PTHREAD_SEM_ERROR_ALREADY_LOCKED;
11    }
12    sem->is_initialized = false;
13    atomic_store(&(sem->value), 0);
14    sem->thread_list.head = nullptr;
15    sem->thread_list.tail = nullptr;
16    return 0;
17 }

```

Détails :

- On vérifie que le sémaphore est initialisé.
- On refuse la destruction si des threads sont encore bloqués dans la file d'attente.
- On remet tous les champs à leur état initial.

Q.6 : pthread_sem_trywait

Rôle : Tente de décrémenter le sémaphore de manière **non bloquante**. Si la valeur est 0, retourne une erreur au lieu de bloquer.

```

1 int pthread_sem_trylock(pthread_sem_t *sem) {
2     if (sem == nullptr) {
3         return PTHREAD_SEM_ERROR_NULL;
4     }
5     if (!sem->is_initialized) {
6         return PTHREAD_SEM_ERROR_INIT;
7     }
8
9     int current = atomic_load(&(sem->value));
10    if (current > 0) {
11        atomic_fetch_sub(&(sem->value), 1);
12        return 0;
13    }
14 }

```

```

15  return MTHREAD_SEM_ERROR_ALREADY_LOCKED;
16  }

```

Détails :

- On lit la valeur courante du sémaphore.
- Si `value > 0` : on décrémente et on retourne 0 (succès).
- Si `value == 0` : on retourne `MTHREAD_SEM_ERROR_ALREADY_LOCKED` sans bloquer le thread. C'est la différence fondamentale avec `sem_wait`.

Q.7 : `pthread_sem_getvalue`

Rôle : Récupère la valeur courante du sémaphore.

```

1  int pthread_sem_getvalue(pthread_sem_t *sem, int *val) {
2      if (sem == nullptr || val == nullptr) {
3          return MTHREAD_SEM_ERROR_NULL;
4      }
5      if (!sem->is_initialized) {
6          return MTHREAD_SEM_ERROR_INIT;
7      }
8      *val = atomic_load(&(sem->value));
9      return 0;
10 }

```

Détails :

- On vérifie que ni `sem` ni `val` ne sont NULL.
- On vérifie que le sémaphore est initialisé.
- On lit la valeur atomiquement et on l'écrit dans `*val`.

3 Démonstration

Q.8 : Programmes d'exemple testant chaque fonction

Test de `pthread_sem_init`

```

1  // Verifie : init normal, init avec valeur negative, double init
2  pthread_sem_t sem;
3  int res;
4
5  res = pthread_sem_init(&sem, 1);    // Doit retourner 0
6  assert(res == 0);
7
8  pthread_sem_t sem2;
9  res = pthread_sem_init(&sem2, -1);  // Doit retourner
10                                     MTHREAD_SEM_ERROR_INIT
11  assert(res == MTHREAD_SEM_ERROR_INIT);
12
13  res = pthread_sem_init(&sem, 1);    // Double init ->
14                                     MTHREAD_SEM_ERROR_INIT
15  assert(res == MTHREAD_SEM_ERROR_INIT);

```

Test de `pthread_sem_wait`

```
1 // Verifie : wait decremente la valeur
2 pthread_sem_t sem;
3 int res, val;
4
5 pthread_sem_init(&sem, 2);
6 pthread_sem_wait(&sem);
7 pthread_sem_getvalue(&sem, &val);
8 assert(val == 1); // 2 - 1 = 1
9
10 pthread_sem_wait(&sem);
11 pthread_sem_getvalue(&sem, &val);
12 assert(val == 0); // 1 - 1 = 0
```

Test de `pthread_sem_post`

```
1 // Verifie : post incremente la valeur
2 pthread_sem_t sem;
3 int res, val;
4
5 pthread_sem_init(&sem, 0);
6 pthread_sem_post(&sem);
7 pthread_sem_getvalue(&sem, &val);
8 assert(val == 1); // 0 + 1 = 1
9
10 pthread_sem_post(&sem);
11 pthread_sem_getvalue(&sem, &val);
12 assert(val == 2); // 1 + 1 = 2
```

Test de `pthread_sem_trywait`

```
1 // Verifie : trylock non bloquant
2 pthread_sem_t sem;
3 int res;
4
5 pthread_sem_init(&sem, 1);
6 res = pthread_sem_trylock(&sem);
7 assert(res == 0); // Succes, valeur passe de 1 a 0
8
9 res = pthread_sem_trylock(&sem);
10 assert(res == EBUSY); // Echec, valeur == 0
11
12 pthread_sem_post(&sem); // valeur remonte a 1
13 res = pthread_sem_trylock(&sem);
14 assert(res == 0); // Succes a nouveau
```

Test de `pthread_sem_getvalue`

```
1 // Verifie : lecture de la valeur et gestion des erreurs
2 pthread_sem_t sem;
3 int res, val;
4
```

```
5 mthread_sem_init(&sem, 42);
6 res = mthread_sem_getvalue(&sem, &val);
7 assert(res == 0 && val == 42);
8
9 res = mthread_sem_getvalue(nullptr, &val);
10 assert(res == MTHREAD_SEM_ERROR_NULL); // sem NULL
11
12 res = mthread_sem_getvalue(&sem, nullptr);
13 assert(res == MTHREAD_SEM_ERROR_NULL); // val NULL
```

Test multi-thread (wait bloquant + post)

```
1 // Verifie le comportement bloquant de sem_wait en multi-thread
2 mthread_sem_t sem;
3
4 void* thread_post(void* arg) {
5     mthread_sem_post(&sem);
6     return nullptr;
7 }
8
9 void* thread_wait(void* arg) {
10     mthread_sem_wait(&sem); // Bloque si value == 0
11     mthread_sem_wait(&sem); // Attend le post
12     return nullptr;
13 }
14
15 int main() {
16     mthread_sem_init(&sem, 1);
17     // Valeur initiale = 1
18     // thread_post fait +1 -> value = 2
19     // thread_wait fait -1, -1 -> value = 0
20     mthread_t t1 = mthread_create_thread(nullptr, thread_post, nullptr);
21     mthread_t t2 = mthread_create_thread(nullptr, thread_wait, nullptr);
22     mthread_join(t1, nullptr);
23     mthread_join(t2, nullptr);
24
25     int val;
26     mthread_sem_getvalue(&sem, &val);
27     assert(val == 0); // 1 + 1 - 1 - 1 = 0
28 }
```